SIMATS ENGINEERING
ASSESSMENT TOOL 1
CO4 – SECURITY TESTING SIMULATION TASK

Course: Ethical Hacking (ITA1407)
Student Name: Pranov Raaj G
Register Number: 192372355
CO4: Implement network security testing methods using Kali Linux.

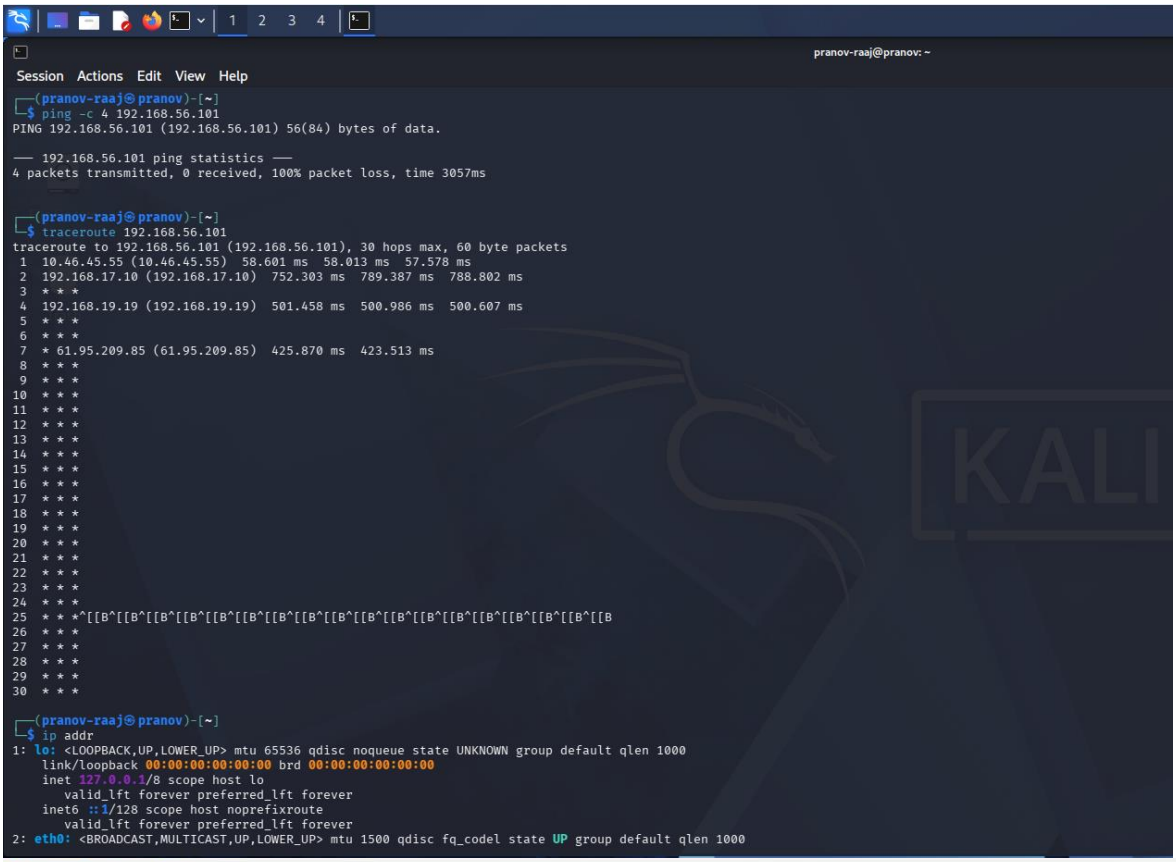
Question 1: Web Server Reconnaissance and Vulnerability Analysis

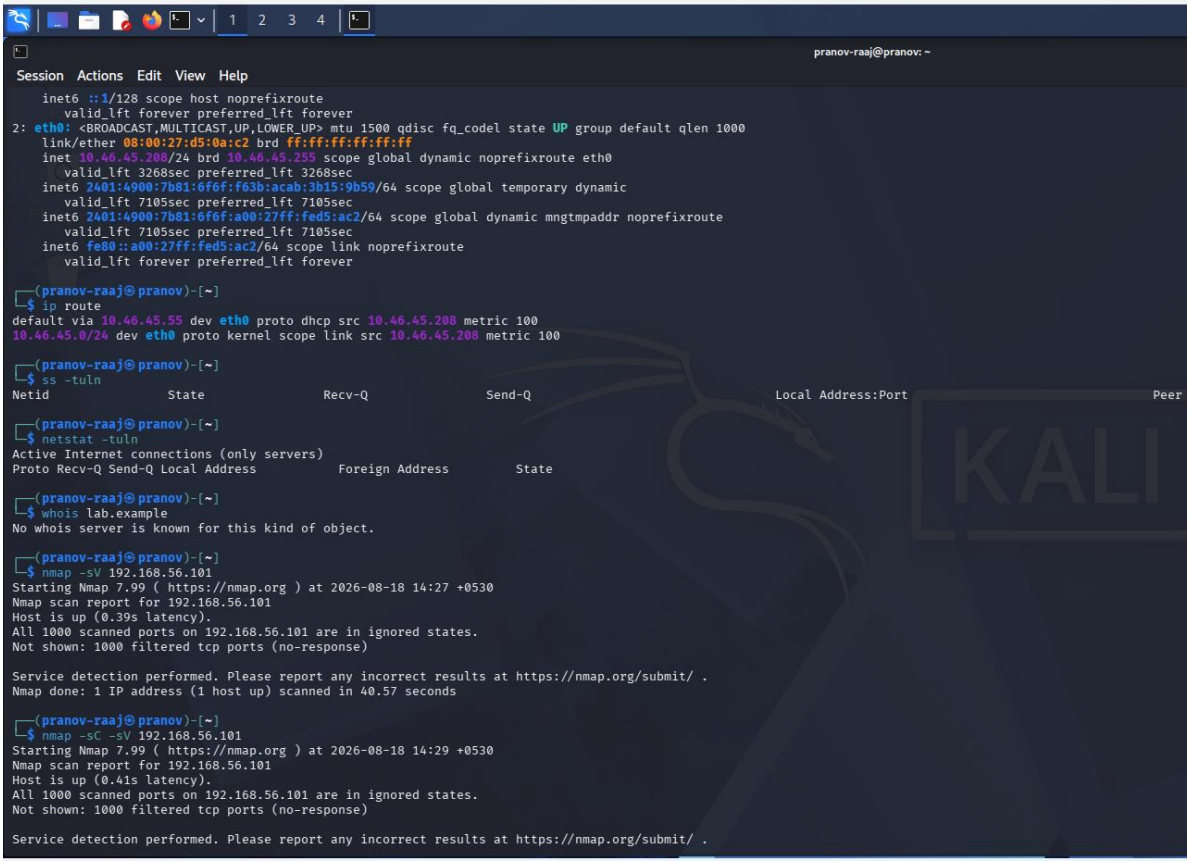
Objective: Perform authorized reconnaissance against the isolated laboratory web server, identify ports/services/technologies, inspect HTTP headers, discover web resources, and classify findings.

Target placeholder: 192.168.56.101 (replace with the faculty-approved lab target).

Commands / Code

```
# Basic network reconnaissance
ping -c 4 192.168.56.101
traceroute 192.168.56.101
ip addr
ip route
ss -tuln
netstat -tuln

# WHOIS / domain information (only for the authorized lab domain)
whois lab.example

# Nmap service and version detection
nmap -sV 192.168.56.101
nmap -sC -sV 192.168.56.101

# Web technology identification
whatweb http://192.168.56.101

# HTTP response headers
curl -I http://192.168.56.101
wget --server-response --spider http://192.168.56.101

# Web-server vulnerability scan
nikto -h http://192.168.56.101

# Directory/file discovery – use only the approved wordlist and lab target
gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt
# Alternative:
dirb http://192.168.56.101
```


1 2 3 4
pranov-raaj@pranov: ~
Session Actions Edit View Help
(pranov-raaj㉿pranov)-[~]
$ ping -c 4 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
— 192.168.56.101 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3057ms
(pranov-raaj㉿pranov)-[~]
$ traceroute 192.168.56.101
traceroute to 192.168.56.101 (192.168.56.101), 30 hops max, 60 byte packets
1 10.46.45.55 (10.46.45.55) 58.601 ms 58.013 ms 57.578 ms
2 192.168.17.10 (192.168.17.10) 752.303 ms 789.387 ms 788.802 ms
3 * * *
4 192.168.19.19 (192.168.19.19) 501.458 ms 500.986 ms 500.607 ms
5 * * *
6 * * *
7 * 61.95.209.85 (61.95.209.85) 425.870 ms 423.513 ms
8 * * *
9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *
25 * * *^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *
(pranov-raaj㉿pranov)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000


1 2 3 4
pranov-raaj@pranov: ~
Session Actions Edit View Help
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:d5:0a:c2 brd ff:ff:ff:ff:ff:ff
inet 10.46.45.208/24 brd 10.46.45.255 scope global dynamic noprefixroute eth0
valid_lft 3268sec preferred_lft 3268sec
inet6 2401:4900:7b81:6f6f:f63b:acab:3b15:9b59/64 scope global temporary dynamic
valid_lft 7105sec preferred_lft 7105sec
inet6 2401:4900:7b81:6f6f:a00:27ff:fed5:ac2/64 scope global dynamic mngtmpaddr noprefixroute
valid_lft 7105sec preferred_lft 7105sec
inet6 fe80::a00:27ff:fed5:ac2/64 scope link noprefixroute
valid_lft forever preferred_lft forever
(pranov-raaj㉿pranov)-[~]
$ ip route
default via 10.46.45.55 dev eth0 proto dhcp src 10.46.45.208 metric 100
10.46.45.0/24 dev eth0 proto kernel scope link src 10.46.45.208 metric 100
(pranov-raaj㉿pranov)-[~]
$ ss -tuln
Netid State Recv-Q Send-Q Local Address:Port Peer
(pranov-raaj㉿pranov)-[~]
$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
(pranov-raaj㉿pranov)-[~]
$ whois lab.example
No whois server is known for this kind of object.
(pranov-raaj㉿pranov)-[~]
$ nmap -sV 192.168.56.101
Starting Nmap 7.99 (https://nmap.org) at 2026-08-18 14:27 +0530
Nmap scan report for 192.168.56.101
Host is up (0.39s latency).
All 1000 scanned ports on 192.168.56.101 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 40.57 seconds
(pranov-raaj㉿pranov)-[~]
$ nmap -sC -sV 192.168.56.101
Starting Nmap 7.99 (https://nmap.org) at 2026-08-18 14:29 +0530
Nmap scan report for 192.168.56.101
Host is up (0.41s latency).
All 1000 scanned ports on 192.168.56.101 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .

Observations

- Record reachable host status and basic network information.
- Record open ports, detected services, and service versions from Nmap.
- Record technologies/frameworks identified by WhatWeb.
- Review HTTP headers for server/version disclosure and missing security controls.
- Review Nikto and Gobuster/Dirb results for potentially exposed files, directories, backup files, or administrative resources.

Findings and Risk Analysis

Potential findings include unnecessary open ports, exposed service versions, server-banner information disclosure, missing security headers, accessible administrative directories, backup/configuration files, and outdated web-server components. Severity should be assigned only after manually validating each finding in the authorized lab.

Recommendations

- Close or firewall unnecessary ports and services.
- Disable unnecessary server/version banners.
- Remove backup, temporary, and configuration files from web-accessible locations.
- Restrict administrative interfaces using authentication and network controls.
- Apply current security patches and harden the web-server configuration.
- Add appropriate security headers and minimize information disclosure.

Question 2: Web Application Vulnerability Assessment

Objective: Assess an authorized vulnerable web application for common web vulnerabilities using Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools.

Target placeholder: http://192.168.56.101/app (replace with the faculty-approved lab URL).

Commands / Code

```
# Reconnaissance
nmap -sV 192.168.56.101
whatweb http://192.168.56.101/app
curl -I http://192.168.56.101/app

# Nikto
nikto -h http://192.168.56.101/app

# OWASP ZAP command-line baseline scan (authorized lab only)
zaproxy -cmd -quickurl http://192.168.56.101/app -quickout /tmp/zap-report.html

# Example safe SQL injection validation string for a TEST input
# Enter in the application's lab login/search parameter:
' OR '1'='1

# Example XSS validation string for a TEST input
<script>alert('XSS')</script>
```

```
# Example directory traversal validation for a deliberately vulnerable lab parameter
../../../../etc/passwd

# Inspect cookies and response headers
curl -i http://192.168.56.101/app
curl -c cookies.txt -b cookies.txt http://192.168.56.101/app
```

Observations

- Map application pages, parameters, forms, cookies, sessions, and API requests.
- Capture requests/responses using Burp Suite or OWASP ZAP.
- Check whether test inputs are safely handled and encoded.
- Check authentication and session-management behavior using only supplied lab accounts.
- Record confirmed findings and distinguish them from automated-tool false positives.

Findings and Severity

- SQL Injection: High/Critical if confirmed because attacker-controlled input can alter database queries.
- Stored/Reflected XSS: Medium/High depending on execution context and affected users.
- Weak authentication/session management: Medium/High depending on exploitability and account impact.
- Directory traversal: High if arbitrary sensitive files can be read.
- Security misconfiguration: severity depends on the exposed information or functionality.

Recommendations

- Use parameterized queries/prepared statements for database access.
- Apply context-aware output encoding and a strong Content-Security-Policy.
- Use secure, HttpOnly, SameSite cookies and rotate sessions after authentication.
- Enforce server-side authorization and input validation.
- Remove unnecessary services and secure application configuration.

Question 3: Security Header and HTTPS Configuration Assessment

Objective: Examine HTTP security headers, HTTPS redirection, TLS protocols/ciphers, and certificate configuration for the authorized laboratory web application.

Target placeholder: 192.168.56.101 (replace with the faculty-approved lab target).

Commands / Code

```
# Inspect HTTP headers
curl -I http://192.168.56.101

# Follow redirects and verify HTTPS enforcement
curl -I -L http://192.168.56.101

# Inspect HTTPS headers
curl -k -I https://192.168.56.101

# Nmap TLS/SSL enumeration
nmap --script ssl-enum-ciphers -p 443 192.168.56.101
nmap --script ssl-cert -p 443 192.168.56.101

# Inspect certificate details
```

```
openssl s_client -connect 192.168.56.101:443 -servername lab.example </dev/null

# Test supported TLS versions in the authorized lab
openssl s_client -connect 192.168.56.101:443 -tls1_2 </dev/null
openssl s_client -connect 192.168.56.101:443 -tls1_3 </dev/null
```

Observations

- Check for Content-Security-Policy (CSP), Strict-Transport-Security (HSTS), X-Frame-Options, and X-Content-Type-Options.
- Verify that HTTP traffic is redirected to HTTPS.
- Record weak TLS protocols, insecure cipher suites, certificate errors, or expired/mismatched certificates if present.
- Record the exact evidence for each confirmed configuration issue.

Risk Analysis

- Missing HSTS can allow downgrade or insecure first-connection scenarios.
- Missing CSP can increase the impact of some XSS attacks.
- Missing clickjacking protection can expose sensitive pages to framing attacks.
- Weak TLS protocols/ciphers can reduce confidentiality and integrity of communications.
- Certificate validation problems can undermine trust in the HTTPS connection.

Recommendations

- Enable HSTS after confirming the HTTPS deployment is correct.
- Deploy a restrictive CSP appropriate to the application.
- Enable X-Frame-Options or frame-ancestors in CSP and X-Content-Type-Options: nosniff.
- Disable obsolete TLS versions and weak cipher suites.
- Use a valid certificate with correct hostname, trust chain, and renewal process.

Question 4: Broken Access-Control Assessment

Objective: Use separate faculty-provided student and administrator test accounts to determine whether authorization is correctly enforced for student records and administrator-only resources.

Commands / Code

```
# Start OWASP ZAP in the laboratory
zaproxy

# Example request inspection using Curl after obtaining a LAB session cookie
curl -i -H "Cookie: SESSION=<LAB_STUDENT_SESSION>"
http://192.168.56.101/student/record?id=1001

# Compare with another faculty-provided TEST object identifier
curl -i -H "Cookie: SESSION=<LAB_STUDENT_SESSION>"
http://192.168.56.101/student/record?id=1002

# Test an administrator-only endpoint with the student test session
curl -i -H "Cookie: SESSION=<LAB_STUDENT_SESSION>"   http://192.168.56.101/admin

# Repeat the same request with the faculty-provided administrator test session
curl -i -H "Cookie: SESSION=<LAB_ADMIN_SESSION>"   http://192.168.56.101/admin
```

Testing Procedure

- Log in with the student test account and capture the request in OWASP ZAP.
- Identify object identifiers in URLs, forms, cookies, or API requests.
- Change only the test identifier supplied by the faculty and compare the server response.
- Attempt access to an administrator-only resource with the student test session.
- Compare HTTP status codes, response bodies, redirects, and returned data.
- Do not use real users, real credentials, or identifiers outside the approved laboratory scope.

Findings and Risk

- IDOR/BOLA is confirmed if a student can access another student's record by changing an object identifier.
- Vertical privilege escalation is confirmed if a normal student account can access administrator-only functionality.
- A 401 response generally indicates missing/invalid authentication, while a 403 response commonly indicates authentication succeeded but authorization failed.
- Severity increases when unauthorized access exposes sensitive records or privileged operations.

Recommendations

- Perform server-side authorization checks for every protected object and operation.
- Validate the authenticated user's role and ownership before returning data.
- Do not rely on hidden UI controls for authorization.
- Use secure object-reference mechanisms and avoid exposing predictable identifiers where practical.
- Log and monitor denied authorization attempts.

Question 5: Automated Scanning and Manual Validation

Objective: Compare automated findings from Nikto and OWASP ZAP, categorize the results, and manually validate at least three safe findings within the authorized laboratory environment.

Commands / Code

```
# Nikto web-server scan
nikto -h http://192.168.56.101 -output nikto-report.txt

# OWASP ZAP baseline scan
zaproxy -cmd -quickurl http://192.168.56.101   -quickout zap-report.html

# Nmap service/version information for correlation
nmap -sV 192.168.56.101

# Manual validation example 1: security headers
curl -I http://192.168.56.101

# Manual validation example 2: exposed common resource
curl -i http://192.168.56.101/robots.txt

# Manual validation example 3: application input (lab-only)
curl -i "http://192.168.56.101/search?q=%3Cscript%3Ealert('XSS')%3C%2Fscript%3E"

# Save a simple command transcript
script -a co4-at1-terminal.txt
```

Comparison and Validation

- Categorize findings into misconfiguration, information disclosure, authentication/session issues, and injection issues.
- For each selected finding, reproduce the behavior manually using a safe request.
- Mark each result as Confirmed, False Positive, or Requires Further Investigation.
- Do not treat automated scanner output alone as proof of a vulnerability.
- Assign severity using likelihood and potential impact.

Sample Findings Table

Finding	Category	Validation	Severity	Remediation
Server/version disclosure	Information disclosure	Confirmed with curl -I	Low	Reduce server banner information
Missing security header	Misconfiguration	Confirmed with curl -I	Low/Medium	Configure required security headers
Injection indicator	Injection	Requires manual lab validation	High if confirmed	Use parameterized queries / output encoding

Overall Conclusion

The assessment follows the CO4 requirement to implement network security testing methods using Kali Linux. The work covers web-server reconnaissance, web-application vulnerability assessment, HTTPS/security-header testing, broken access-control testing, and automated scanning with manual validation. All commands and tests must be executed only against the faculty-approved isolated laboratory targets. Findings should be confirmed manually before assigning severity, and practical remediation should be documented for every confirmed issue.

Student: Pranov Raaj G | Reg. No.: 192372355